COMSATS UNIVERSITY ISLAMABAD ATTOCK CAMPUS

Lab MidTerm

Course name: Programming for AI **Course Code: AIC270**

Instructor: Ms. Fouzia Jabeen **Marks: 30**

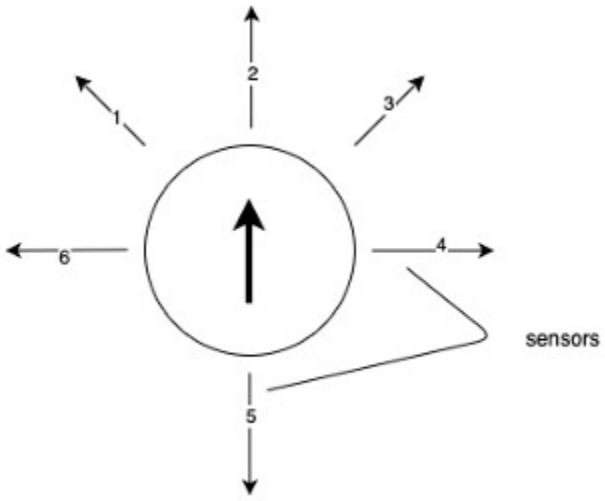
[CLO-4]: Implement AI based applications and optimization techniques using Python. BTL=[Applying]

Q No 01:

Design a robotic controller using computational intelligence techniques. Robots are often required to perform complex behaviors such as autonomous navigation—moving from one point to another within an environment, for example, transporting goods in a warehouse. The robot is equipped with sensors for environmental perception and wheels for movement.

Explain how sensor data can be linked to motor actions to enable navigation. Instead of using neural networks or reinforcement learning, apply a Genetic Algorithm (GA) approach. Describe how the GA can be used to evolve and evaluate controllers based on a fitness function, and discuss how this process can be implemented and tested using a virtual robot in a simulated environment.

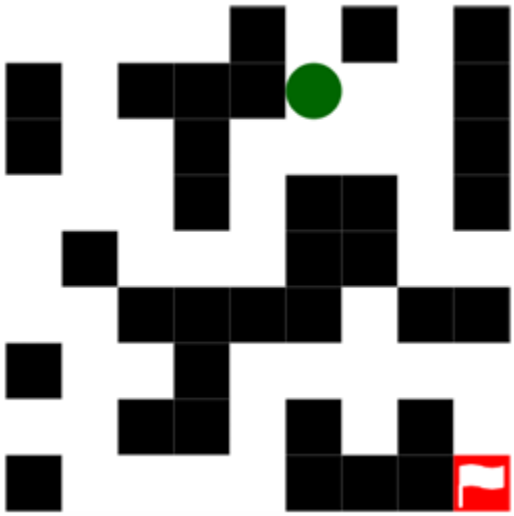
The target: Our robot can take five actions: move one step forward,move one step back, turn left, turn righ and do nothing. The robot also has six sensors in the figure below.



- three on the front

- one on the left
- one on the right
- one on the back

The maze is comprised of walls that the robot can’t cross and will have an outlined route. We are going to design a robotic controller that can use the robot sensors to navigate a robot successfully through a maze.



Mapping the sensor values to actions: As mentioned in the previous section, the robot will have five actions that can be represented in binary as follows:

- “000” — do nothing;
- “001” — move forward;
- “010” — move back;
- “011” — turn left;
- “100” — turn right.

Since five actions cannot be represented with 2 bits, each action now requires 3 bits ($2^3 = 8$ codes). The remaining codes “101”, “110” and “111” are unused and are treated as invalid (the GA must not generate them, or they are decoded as “do nothing”).

We also have six different sensors. To simplify the representation we limit the measurements to binary encoding, that is, values less than a threshold indicate obstacle detection and larger than threshold indicate a clear path. 6 sensors are giving us $2^6 = 64$ possible combinations of sensor inputs. Since an action requires 2 bits, our controller requires 64*3 = 192 bits of storage to represent any possible input. Given that 192 bits are our requirement for representing instructions different combinations. But how should we organize the chromosome so we can encode and decode it?

We have a human readable list of inputs and outputs as follows:

- Sensor #1 (front): on
- Sensor #2 (front-left): off
- Sensor #3 (front-right): on
- Sensor #4 (left): off
- Sensor #5 (right): off
- Sensor #6 (back): off

We also have an instruction that is “turn left” with a value of 011 in binary. Our next is to take the six sensor values and encode those further.

000101 => 011

If we now convert the sensor values’ bit string to decimal, we get the following:

5 => 011

So, we can use the sensor's decimal value as the position in the chromosome that represents a combination of sensor inputs as follows.

xxx xxx xxx xxx xxx 011 xxx xxx xxx xxx (… 54 more triplets…)

As can be seen in the following figure, a combination of sensor values produces a 3 bit binary output that describes how a typical chromosome can map the robot's sensor values to actions.

Sensors						Action	Chromosome (3 bits per action)
B	R	L	FR	FL	F		
0	0	0	0	0	0	100	**100**000011001011011010...100
0	0	0	0	0	1	000	100**000**011001011011010...100
0	0	0	0	1	0	011	100000**011**001011011010...100
0	0	0	0	1	1	001	100000011**001**011011010...100
0	0	0	1	0	0	011	100000011001**011**011010...100
0	0	0	1	0	1	011	100000011001011**011**010...100
0	0	0	1	1	0	010	100000011001011011**010**...100
1	1	1	1	1	1	100	100000011001011011010...**100**